VERIQTA

50 AWS Interview Scenarios

Real-World Questions, Architecture Decisions and Production Fixes



- Architecture and Design Decisions
- Troubleshooting Real AWS Incidents
- Security, IAM and Access Control
- High Availability and Disaster Recovery
- Performance and Scaling
- Cost Optimization and Governance
- CI/CD and Infrastructure as Code
- End-to-End Production Scenarios

@veriqta Instagram | @veriqta GitHub | @veriqta LinkedIn | @veriqta Telegram | @veriqta X | @veriqta Discord

VERIQTA

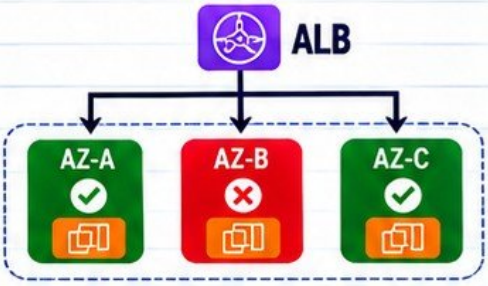
The VPC Is Healthy, but Production Traffic Is Disappearing

Scenario 01

Your application runs across three Availability Zones behind an Application Load Balancer. One AZ loses outbound connectivity, but the load balancer still considers its targets healthy. How would you identify the actual failure domain without unnecessarily evacuating the entire AZ?

Key Investigation Areas

- Verify outbound connectivity per AZ (e.g., curl, traceroute, VPC Flow Logs).
- Check NAT gateway, route tables, and NACLs for the affected AZ.
- Compare target health vs. actual egress reachability.



Expected Outcome

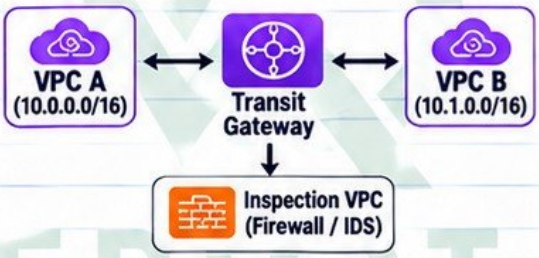
- Identify the specific AZ egress failure.
- Confirm the root cause (NAT, routing, or network policy).
- Keep healthy AZs serving traffic while fixing the failed AZ.

Scenario 02

Two VPCs communicate through Transit Gateway, Connections work in one direction, but return traffic disappaars after a new inspection VPC is introduced. How would you investigate asymmetric routing, stateful firewall behavior, and route-table associations?

Key Investigation Areas

- Validate Transit Gateway route tables and associations.
- Check routing symmetry (forward and return paths).
- Review inspection VPC/firewall stateful behavior and session tracking.



Expected Outcome

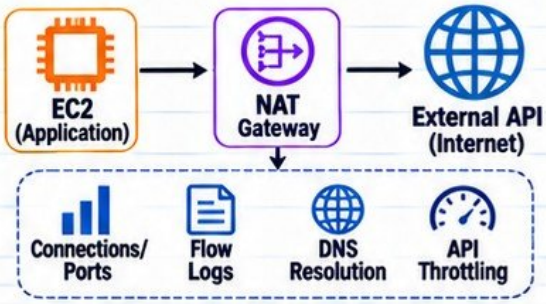
- Identify the asymmetric routing or policy causing one-way traffic.
- Confirm if the inspection device i dropping return sessions.
- Restore symmetric routing and validate end-to-end connectivity.

Scenario 03

An EC2 application intermittently fails to reach an external API. Security groups, NACLs, and route tables appear correct. How would you distinguish NAT gateway connection exhaustion, ephemeral-port pressure, DNS failures, and upstream throttling?

Key Investigation Areas

- Check NAT gateway metrics (connection count, port usage).
- Review VPC Flow Logs for drops or resets.
- Validate DNS resolution (e.g., dig, nslookup, resolver logs).
- Confirm upstream API limits (rate limits, 429/5xx responses).



Expected Outcome

- Identify the real cause: NAT exhaustion, port pressure, DNS failure, or API throttling.
- Apply the right fix (e.g., add NAT capacity, adjust port/connection limits, fix DNS, or implement backoff/retries).

Senior Interview Focus: Network evidence, failure isolation, routing symmetry, and safe mitigation.

VERIQTA

DNS Failures That Look Like Application Bugs

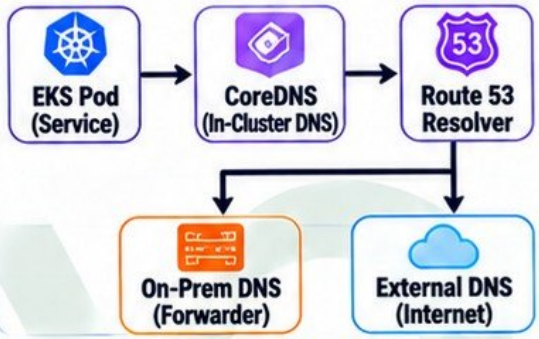
Scenario 04

A production EKS cluster experiences intermittent service discovery failures after hybrid DNS forwarding rules are updated. Only some workloads fail. How would you investigate resolver paths, CoreDNS behavior, caching, and forwarding loops?

Key Investigation Areas

- Verify DNS resolution path (pod → CoreDNS → Route 53 Resolver → on-prem).
- Inspect CoreDNS configuration, logs, and error rates.
- Check forwarding rules for loops or conflicting entries.
- Validate DNS caching (TTL) and stale records.

EKS DNS Resolution Flow



Expected Outcome

- Identify where resolution breaks or becomes inconsistent.
- Confirm CoreDNS health and forwarding behavior.
- Detect and fix forwarding loops or misconfigured rules.
- Ensure stable, low-latency service discovery for all pods.

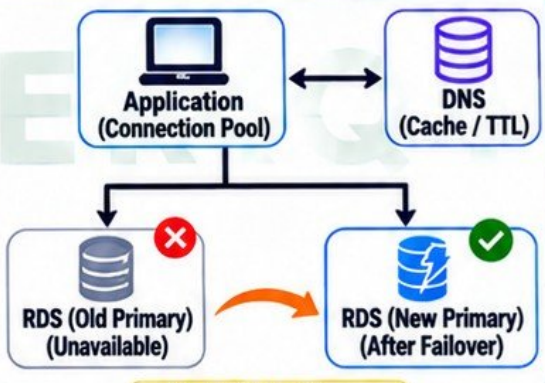
Scenario 05

An RDS Multi-AZ failover completes successfully, but application instances continue connecting to the former database endpoint address. How would you diagnose DNS caching, stale connections, connection pools, and application recovery behavior?

Key Investigation Areas

- Verify the RDS failover event and new primary endpoint.
- Check DNS TTL and caching on OS, JVM, or application resolver.
- Inspect connection pool behavior (idle/long-lived connections).
- Validate retry and reconnection logic in the application.

RDS Multi-AZ Failover Flow



Multi-AZ Failover

Expected Outcome

- Confirm DNS updated to the new RDS endpoint.
- Identify stale DNS cache or long-lived connections blocking recovery.
- Adjust TTL, flush cache, and renew connections safely.
- Ensure the application reconnects and fully recovers after failover.

Senior Interview Focus:

DNS resolution paths, cache behavior, dependency recovery, and time-correlated evidence.

VERIQTA

The IAM Policy That Brought Down Production

Scenario 06

A new service control policy is deployed through AWS Organizations. Several production Lambda functions immediately begin receiving AccessDenied errors, even though their IAM roles have not changed. How would you identify the effective permission boundary and safely restore service?

Key Investigation Areas

- Check the effective permission flow: SCP → Permission Boundary → IAM Policy.
- Review AWS Organizations SCP (new/updated) and its scope (OUs and accounts).
- Analyze CloudTrail and Lambda error logs (AccessDenied details).

Permission Evaluation Flow



Effective permission = SCP ∩ Boundary ∩ IAM Policy

Expected Outcome

- Identify the restrictive SCP statement causing denial.
- Confirm the affected accounts and Lambda roles.
- Restore service by updating the SCP (scope exception or policy adjustment) safely.

Scenario 07

A CI/CD pipeline assumes a production deployment role through OIDC. An attacker gains control of an untrusted workflow and attempts to use the same role. What conditions, trust policies, and deployment boundaries would you inspect first?

Key Investigation Areas

- Review the IAM role trust policy (oidc:sub, aud, and condition keys).
- Validate the OIDC provider (GitHub Actions, GitLab, etc.) and allowed repositories/branches.
- Check deployment boundaries (least privilege, environment and account separation).

OIDC Trust Flow



Validate: audience (aud), subject (sub), repository, branch, and environment conditions.

Expected Outcome

- Confirm whether the workflow identity meets the trust conditions.
- Identify misconfigurations (e.g., overly broad sub or missing branch restrictions).
- Tighten trust policy and restrict deployments to approved repos, branches, and environments.

Scenario 08

An engineer accidentally exposes an AWS access key in a public repository. The key has already been used to create infrastructure in an unfamiliar Region. What is your containment sequence, and how do you preserve evidence while stopping further misuse?

Key Investigation Areas

- Immediately revoke or disable the exposed access key.
- Use CloudTrail and AWS Config to identify actions, resources, and the Region used.
- Preserve evidence (logs, events, resource history) for investigation.

Containment Sequence



Expected Outcome

- Immediately stop further access and prevent new changes.
- Identify and remove unauthorized resources in the unfamiliar Region.
- Keep a full evidence trail (CloudTrail, Config, and logs) for security review and post-incident fixes.

Senior Interview Focus:

Effective permissions, identity trust, credential containment, and incident authority.

Encryption and Secrets That Become Availability Risks

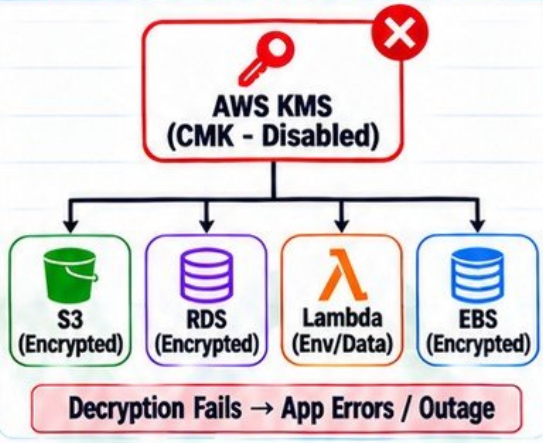
Scenario 09

A customer-managed KMS key is disabled during a security response. Multiple applications can no longer decrypt required data. How would you determine the affected resources, assess the security implications, and choose a safe recovery path?

Key Investigation Areas

- Identify all resources using the KMS key (AWS Config, CloudTrail, KMS key details).
- Check for decryption failures in application logs and CloudWatch metrics.
- Assess the security reason for disabling the key and confirm scope of impact.
- Plan a controlled recovery (re-enable, new key, or re-encrypt).

KMS Dependency Overview



Expected Outcome

- List all affected services and data dependencies.
- Confirm the security risk and business impact (e.g., data access loss).
- Restore service safely (re-enable the key or migrate to a new key).
- Document the change, root cause, and preventive controls (key policies, alerts).

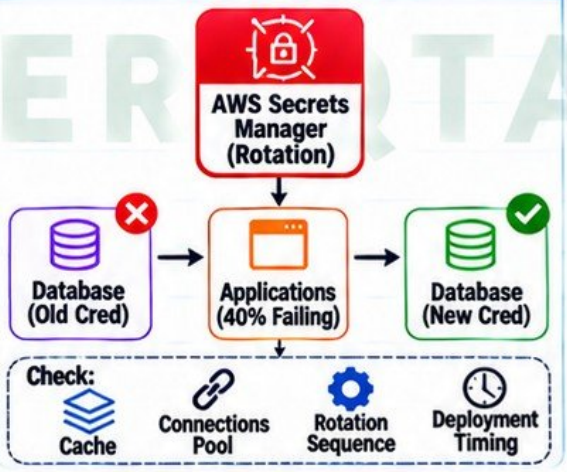
Scenario 10

AWS Secrets Manager rotates a database credential successfully, but 40% of application requests begin failing authentication. How would you investigate cached credentials, existing connections, rotation sequencing, and recovery without exposing secrets?

Key Investigation Areas

- Confirm the rotation event timeline in Secrets Manager and CloudTrail.
- Check application logs for authentication failures (40% pattern indicates partial rollout or caching).
- Investigate credential caching (app config, containers, Lambda environment, connection pools).
- Review rotation strategy (automatic, single-user, multi-database, or custom).

Secret Rotation Flow



Expected Outcome

- Identify why only 40% of requests fail (cached creds, long-lived connections, or staggered rollout).
- Safely refresh credentials (rotate connections, restart services, or update config).
- Ensure zero secret exposure (use Secrets Manager, IAM permissions, and encrypted configuration).
- Validate full recovery and monitor for recurrence.

Senior Interview Focus:

Cryptographic dependency mapping, rotation safety, and security versus availability.

The EC2 Fleet Has Spare CPU, but Users Experience Severe Latency

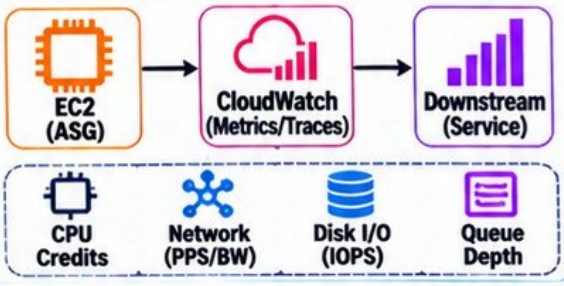
Scenario 11

An Auto Scaling group averages 25% CPU utilization, yet API p99 latency exceeds the service-level objective. How would you investigate CPU credit depletion, network limits, disk I/O, application queueing, and downstream dependencies?

Key Investigation Areas

- Check instance type and CPU credit balance (T3/T4g).
- Validate network performance (bandwidth, PPS, and ENA).
- Review disk I/O (EBS IOPS/ throughput and queue depth).
- Inspect application queueing and downstream service latency (traces, dependency metrics).

Investigation Flow



Expected Outcome

- Identify the real bottleneck (CPU credits, network, disk, queue, or downstream).
- Correlate metrics with p99 latency and user impact.
- Apply the right fix (scale, instance type change, I/O adjustment, or dependency optimization).

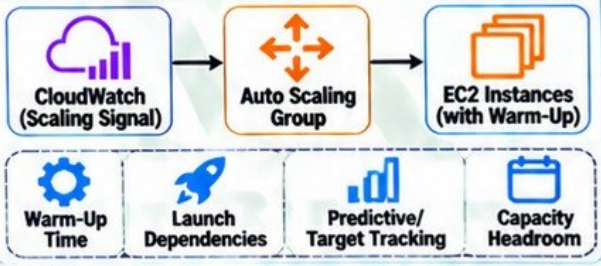
Scenario 12

An Auto Scaling group launches new instances during a traffic spike, but the instances become healthy only after the spike has passed. How would you redesign capacity planning, warm-up behavior, scaling signals, and launch dependencies?

Key Investigation Areas

- Review scaling policies, CloudWatch alarms, and metrics (CPU, RPS, custom).
- Check instance warm-up and health check configuration (ELB/target group).
- Validate launch dependencies (AMI, user data, app bootstrap, external services).

Scaling and Warm-Up Design



Expected Outcome

- Ensure instances are ready before receiving traffic.
- Use the right scaling signal (e.g., request count or latency).
- Include warm-up time and application bootstrap checks.
- Plan for burst capacity (predictive or scheduled scaling) and maintain headroom.

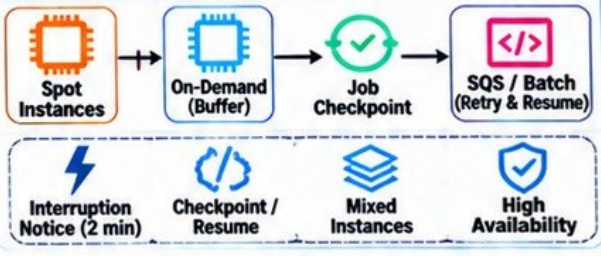
Scenario 13

A fleet uses Spot Instances to reduce costs. A sudden capacity interruption removes most workers while a critical batch job is running. How would you recover processing safely and redesign the fleet to tolerate interruption?

Key Investigation Areas

- Check Spot interruption events and termination notices (EC2 and EventBridge).
- Assess job state (checkpoint, partial progress, and idempotency).
- Review Auto Scaling and capacity strategy (On-Demand fallback and diversification).

Spot Resilience Design



Expected Outcome

- Recover the batch job without data loss or duplication.
- Use checkpoints and idempotent processing.
- Adopt a mixed capacity strategy (Spot + On-Demand) and instance diversification.
- Design for interruption as a normal operational event.

Senior Interview Focus:

Saturation, leading indicators, capacity headroom, and interruption-aware architecture.

VERIQTA

The Instance Is Running, but the Service Is Dead

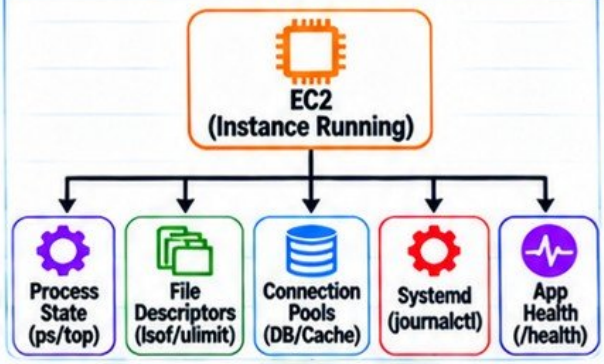
Scenario 14

EC2 status checks pass, but an application is unavailable. CloudWatch CPU and memory appear normal. What would you investigate across process state, file descriptors, connection pools, systemd, and application health?

Key Investigation Areas

- Verify the application process is running and healthy.
- Check file descriptor usage (open files, ulimit).
- Inspect connection pools (db/cache/external APIs).
- Review systemd service state and restart behavior.
- Validate application health (endpoints, logs, error rates).

Investigation Flow



Expected Outcome

- Identify why the service is unavailable despite a healthy instance.
- Find issues such as a crashed/stuck process, FD exhaustion, pool saturation, or misconfigured systemd.
- Restore service safely (restart, tune limits, or fix dependencies).

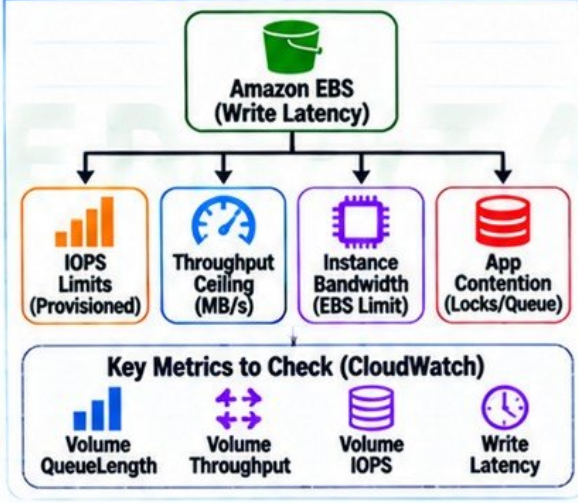
Scenario 15

An EBS-backed application suffers severe write latency after a storage configuration change. How would you distinguish provisioned IOPS limits, throughput ceilings, instance bandwidth constraints, and application-level contention?

Key Investigation Areas

- Check EBS metrics (IOPS, Throughput, QueueLength, VolumeConsumedReadWriteOps).
- Validate volume type and provisioned limits (gp3/io1/io2).
- Review instance EBS bandwidth limits (per instance type).
- Analyze application I/O patterns (fs/DB logs, concurrent writes, locks).
- Correlate timing with the recent configuration change (IOPS/throughput/volume type).

Storage Performance Analysis



Expected Outcome

- Determine the true bottleneck (IOPS limit, throughput, instance cap, or application contention).
- Confirm if the configuration change reduced performance (e.g., gp3 settings, IOPS/MB/s).
- Apply the right fix (increase IOPS/throughput, change instance type, or optimize application I/O).

Senior Interview Focus:

Infrastructure health versus service health, and performance diagnosis through measurable evidence.

VERIQTA

Kubernetes Has Capacity, but Pods Cannot Run

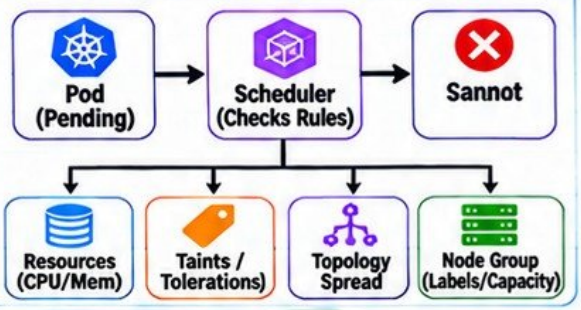
Scenario 16

An EKS cluster has idle nodes, yet critical Pods remain Pending. How would you investigate scheduler constraints, topology spread, taints, resource requests, and node-group configuration?

Key Investigation Areas

- Check Pod events and scheduler messages (kubectl describe podl).
- Verify resource requests (CPU, memory, GPU) vs node capacity.
- Review taints, tolerations, and node selectors.
- Validate topology spread constraints and node-group labels.

Scheduling Diagnostic Flow



Expected Outcome

- Identify the exact scheduling constraint blocking the Pod.
- Fix the constraint (adjust requests, tolerations, or spread rules).
- Confirm the Pod is scheduled and running successfully.
- Prevent recurrence with clear resource and policy standards.

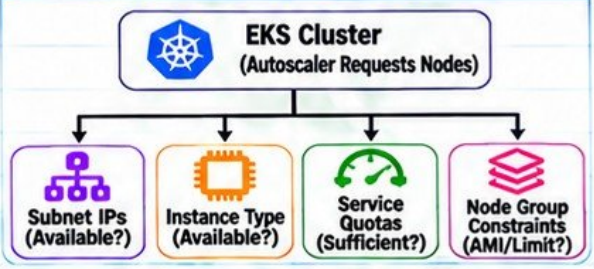
Scenario 17

Cluster Autoscaler requests additional nodes, but Pods remain unschedulable because AWS cannot provision the required capacity. How would you investigate subnet IP availability, instance-type constraints, quotas, and fallback capacity?

Key Investigation Areas

- Check Cluster Autoscaler logs and scale-up events.
- Verify subnet IP availability (used vs. free IPs).
- Confirm instance type availability and EC2 quotas in the target AZ/Region.
- Review node-group constraints (AMI, capacity type, limits).

EKS Scale-Up Investigation



Expected Outcome

- Find the blocking factor (e.g., no free IPs, instance unavailable, or quota limit).
- Resolve by increasing IPs, requesting aa quota increase, changing instance type, or adding a fallback node group (e.g., different type or Spot).
- Confirm autoscaler scales and Pods are scheduled.

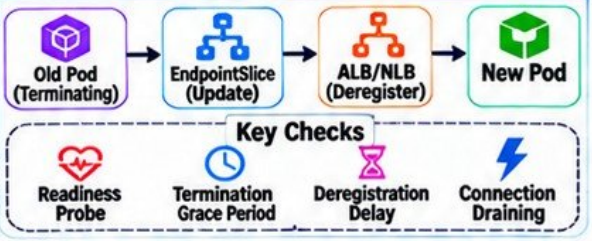
Scenario 18

A rolling deployment causes intermittent 503 errors even though all Pods eventually become Ready. How would you analyze readiness probes, termination behavior, EndpointSlice updates, target deregistration, and connection draining?

Key Investigation Areas

- Review readiness and liveness probe configuration.
- Check Pod terminationGracePeriod an! preStop hooks.
- Inspect EndpointSlice (or Endpoints) update timing.
- Validate ALB/NLB target deregistration and deregistration delay).

Rolling Deployment Flow



Expected Outcome

- Identify the cause of 503 (e.g., early traffic, slow deregistration, or probe misconfiguration).
- Tune probes and termination settings (grace period, preStop).
- Align ALB/NLB deregistration delay with application shutdown.
- Achieve zero or minimal 503s during rolling updates.

Senior Interview Focus:

Scheduling constraints, AWS capacity dependencies, and safe rollout mechanics.

The EKS Cluster Is Healthy, but the Platform Is Failing

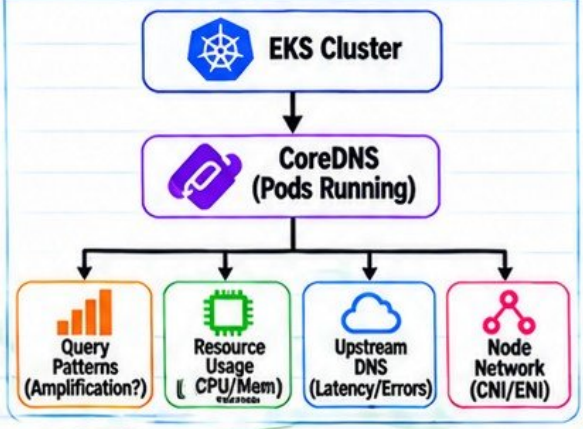
Scenario 19

CoreDNS Pods are running, but DNS latency increases dramatically during a deployment surge. How would you determine whether the cause is query amplification, resource throttling, upstream DNS, or node-level networking?

Key Investigation Areas

- Check CoreDNS metrics (qps, latency, errors, cache hit rate).
- Inspect Pod resource usage (CPU, memory, throttling).
- Analyze DNS query patterns (loops, amplification, or excessive lookups).
- Test upstream DNS (VPC Resolver, Route 53, or on-prem forwarders).
- Validate node networking (DNS, CNI, and ENI usage).

Investigation Flow



Expected Outcome

- Identify the real cause of DNS latency (amplification, throttling, upstream, or network).
- Apply the right fix (tune CoreDNS, increase resources, fix forwarding, or resolve network bottleneck).
- Stabilize DNS latency and maintain performance during deployment surges.

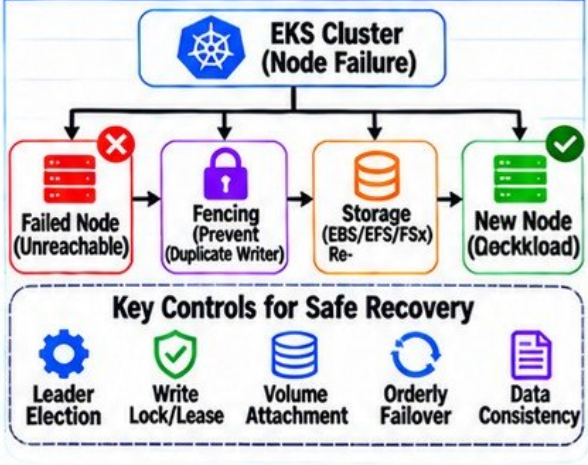
Scenario 20

An EKS node becomes unreachable while hosting stateful workloads. Automated remediation begins replacing it, but the application risks duplicate writers. How would you handle node failure, storage attachment, fencing, and recovery authority?

Key Investigation Areas

- Confirm node failure (not just a network partition) (kubectl, node events, CloudWatch).
- Check stateful workload type (StatefulSet, database, or leader-based application).
- Verify storage stae (EBS/EFS/FSx) and attachment status.
- Confirm fencing/leader election (e.g., lease, quorum, or lock).
- Review remediation behavior (Cluster Autoscaler, node drain, and replacement flow).

Recovery and Fencing Flow



Expected Outcome

- Prevent duplicate writers during node replacement.
- Ensure storage is safely detached and re-attached to the new node.
- Maintain application consistency and data integrity.
- Establish a clear recovery authority (leader election, quorum, or lock).

Senior Interview Focus:

Control-plane versus data-plane evidence, stateful recovery, and preventing duplicate ownership.

The Database Is Available, but Transactions Are Failing

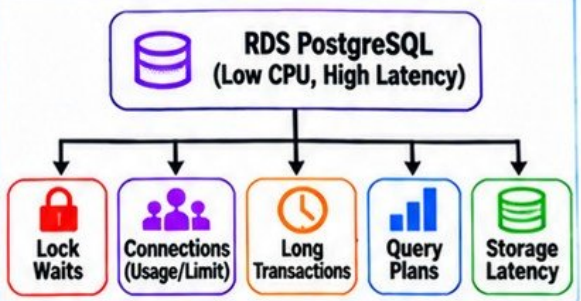
Scenario 21

An RDS PostgreSQL database has low CPU utilization, but transaction latency rises sharply. How would you investigate lock waits, connection saturation, storage latency, long-running transactions, and query plans?

Key Investigation Areas

- Check active sessions and lock waits (pg_stat_activity, pg_locks).
- Verify connection usage (max_connections, pool size).
- Review long-running transactions (duration, state).
- Analyze slow queries and query plans (EXPLAIN ANALYZE).
- Check storage latency (CloudWatch EBS/IO metrics).

PostgreSQL Investigation Flow



Expected Outcome

- Identify the real bottleneck (locks, connections, I/O, or inefficient queries).
- Terminate or tune problem transactions and queries.
- Optimize indexes and queries (based on query plan).
- Adjust connection pool and database parameters.
- Restore normal transaction latency and stability.

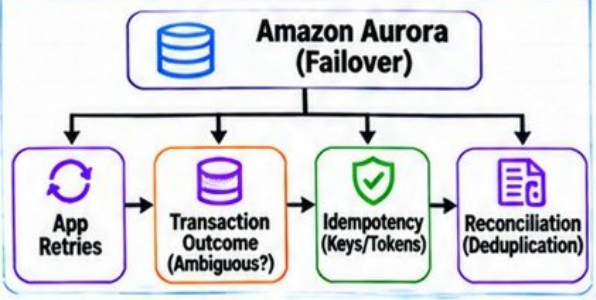
Scenario 22

An Aurora cluster fails over successfully, but the application starts producing duplicate business transactions. How would you investigate retries, transaction outcome ambiguity, idempotency, and application reconciliation?

Key Investigation Areas

- Review application retry logic and timeout settings.
- Check transaction isolation and commit behavior.
- Analyze logs around the failover window (app and DB).
- Validate idempotency controls (request IDs, unique keys).
- Review reconciliation process (detect and handle duplicates).

Aurora Failover and Duplicate Analysis



Expected Outcome

- Confirm whether duplicates were caused by retries during failover.
- Ensure idempotency at the application or API level (e.g., unique request IDs).
- Reconcile and clean up duplicate transactions safely.
- Update retry and timeout strategy to prevent recurrence.

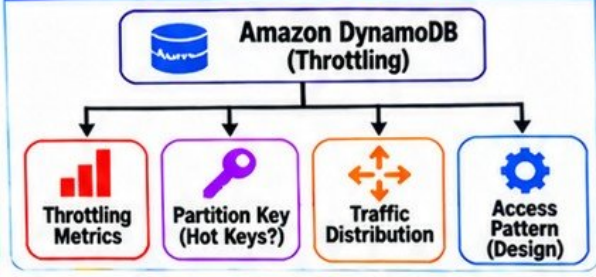
Scenario 23

A DynamoDB table experiences throttling despite sufficient overall provisioned capacity. How would you identify hot partition keys, uneven traffic distribution, and access-pattern design problems?

Key Investigation Areas

- Check throttling metrics (ThrottledRequests, Consumed Read/Write Capacity).
- Analyze access patterns and identify hot partition keys (CloudWatch, Contributor Insights).
- Review key design and traffic distribution (high-cardinality keys).
- Evaluate capacity mode (provisioned vs. on-demand) and partition scaling limits.

DynamoDB Traffic Analysis



Expected Outcome

- Identify hot partition keys causing uneven traffic.
- Redesign keys or isolate high-traffic items (e.g., add key sharding or suffix).
- Adjust capacity (increase, use on-demand, or adaptive scaling).
- Eliminate throttling and stabilize performance.

Senior Interview Focus: Transaction correctness, contention, partition behavior, and recovery from ambiguous outcomes.

Database Scaling That Introduces New Failure Modes

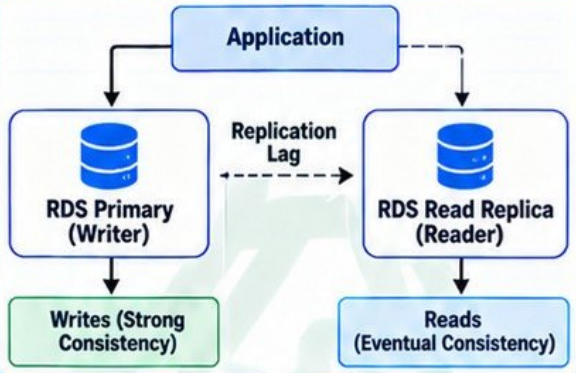
Scenario 24

A company adds RDS read replicas to improve performance. Users begin seeing stale account balances immediately after successful updates. How would you investigate replication lag and design read consistency for sensitive operations?

Key Investigation Areas

- Check replication lag metrics (ReplicaLag, ReplicationLatency, ReplicaStatus).
- Verify read/write routing and connection settings (cluster and endpoint).
- Validate consistency model (read-after-write, session, or eventual consistency).
- Review application logic, caching, and transaction boundaries.

RDS Read Replica Architecture



Expected Outcome

- Identify the replication lag and its root cause (network, load, or configuration).
- Design read consistency for sensitive operations (e.g., account balance, payments).
- Route critical reads to the primary or use session consistency where needed.
- Optimize replica count, placement, and monitoring.

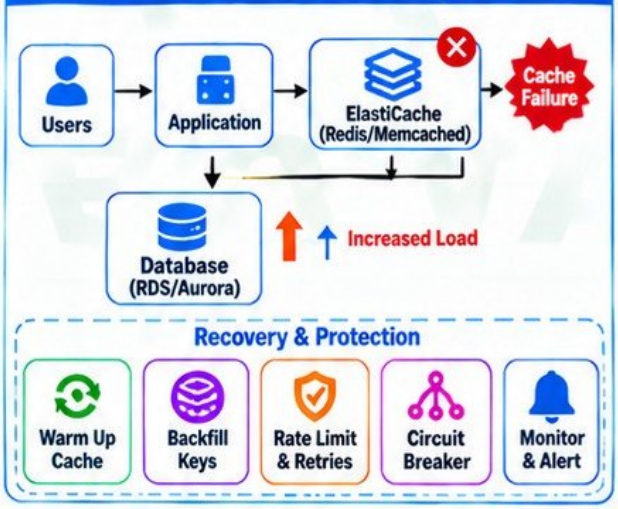
Scenario 25

An ElastiCache cluster fails during peak traffic, causing an unexpected database overload. How would you prevent cache stampedes, coordinate recovery, and preserve servicce availability without introducing incorrect data?

Key Investigation Areas

- Check ElastiCache metrics (hit rate,evictions, CPU, memory, connections).
- Identify failure pattern (node, AZ,or cluster wide).
- Confirm database load increase and query patterns.
- Review cache key design, TTL, and expiration behavior (cache stampede risk).
- Check application error handling, retries, and circuit breakers.

Cache Failure & Recovery Flow



Expected Outcome

- Prevent cache stampedes (using TTLl jitter, warm-up, locking, or request coalescing).
- Isolate cache failure (with circuit breakers and fallback logic).
- Recover the cache cluster safely and rebuild keys without data corruption.
- Protect the database from sudden load spikes and ensure service availability.
- Validate data integrity and monitor closely during recovery.

Senior Interview Focus:

Read-after-write consistency, cache failure isolation, and downstream protection.

Lambda Scaling Is Overwhelming the Rest of AWS

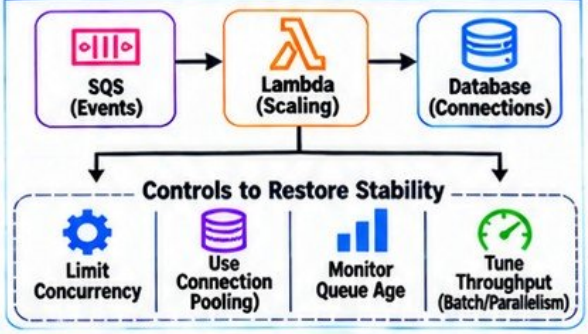
Scenario 26

An SQS-triggered Lambda workload scales rapidly and exhausts available database connections. How would you coordinate event-source concurrency, database connection management, queue age, and processing throughput?

Key Investigation Areas

- Check Lambda concurrency (total, reserved, and event-source).
- Review database connection usage and limits (max_connections, pool size).
- Inspect SQS metrics (ApproximateAgeOfOldestMessage, message backlog).
- Analyze processing duration and throughput (records/sec).

Investigation Flow



Expected Outcome

- Stabilize Lambda concurrency to match database capacity.
- Use connection pooling or RDS Proxy to manage connections efficiently.
- Clear or reduce SQS backlog and keep queue age within SLO.
- Maintain steady processing throughput without overwhelming the database.

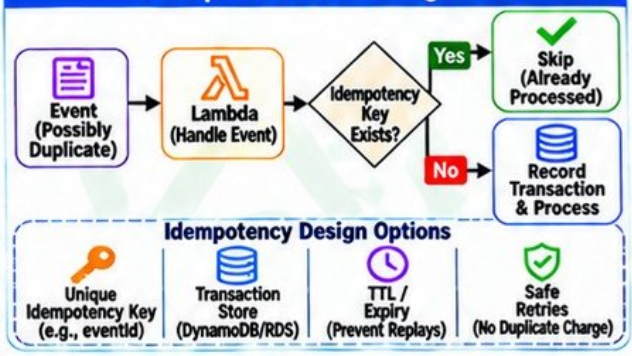
Scenario 27

A Lambda function occasionally runs twice for the same business event. The first execution already charged a customer. How would you design idempotency, transaction records, and recovery from ambiguous outcomes?

Key Investigation Areas

- Confirm event delivery behavior (at-least-once from SQS/EventBridge).
- Check for an idempotency key in the event or request.
- Review transaction and payment records (status, reference IDs).
- Analyze retry behavior, timeouts, and error handling.

Idempotent Processing Flow



Expected Outcome

- Ensure each business event is processed only once (idempotent).
- Use a transaction record with a unique idempotency key and status.
- Handle retries safely and recover from partial or ambiguous outcomes.
- Prevent duplicate charges and maintain data consistency.

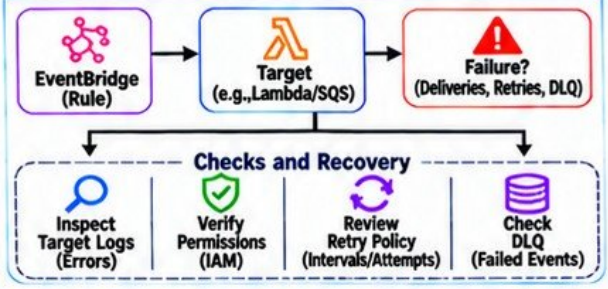
Scenario 28

An EventBridge-based workflow silently stops delivering events to one consumer while other consumers continue operating. How would you investigate target failures, retry policies, dead-letter queues, permissions, and event observability?

Key Investigation Areas

- Check EventBridge rule and target configuration (status, ARN, and health).
- Review target execution logs (Lambda, SQS, or API) for errors.
- Verify retry policy and DLQ (dead-letter queue) settings.
- Confirm IAM permissions (EventBridge → target).
- Analyze event metrics (Matched, Invocations, FailedInvocations).

EventBridge Investigation Flow



Expected Outcome

- Identify why one target stopped receiving events (errors, access, or misconfiguration).
- Restore event delivery and process any messages in the DLQ.
- Ensure correct retry policy and error handling.
- Add monitoring and alerts (e.g., EventBridge metrics and DLQ alarms).

Senior Interview Focus: At-least-once delivery, concurrency control, idempotency, and event recovery.

Asynchronous Systems That Fail Without Visible Errors

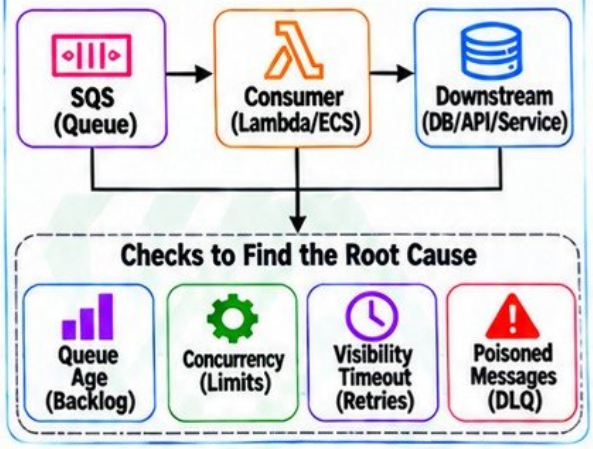
Scenario 29

An SQS queue has growing message age, but consumer CPU remains low. How would you distinguish downstream throttling, visibility-timeout behavior, concurrency limits, and poisoned messages?

Key Investigation Areas

- Check SQS metrics (ApproximateAgeOfOldestMessage, NumberOfMessagesVisible, NumberOfMessagesNotVisible).
- Verify consumer concurrency (Lambda reserved concurrency or ECS service limits).
- Review visibility-timeout and message retry behavior (re-delivery patterns).
- Identify downstream throttling (HTTP 429/5xx, DB limits, rate limits).
- Detect poisoned messages (repeated failures, DLQ, error logs).

SQS Investigation Flow



Expected Outcome

- Identify the real bottleneck (downstream throttling, concurrency, timeout, or bad messages).
- Adjust concurrency and visibility-timeout settings appropriately.
- Quarantine or fix poisoned messages using a DLQ.
- Restore message processing and reduce queue age to normal.

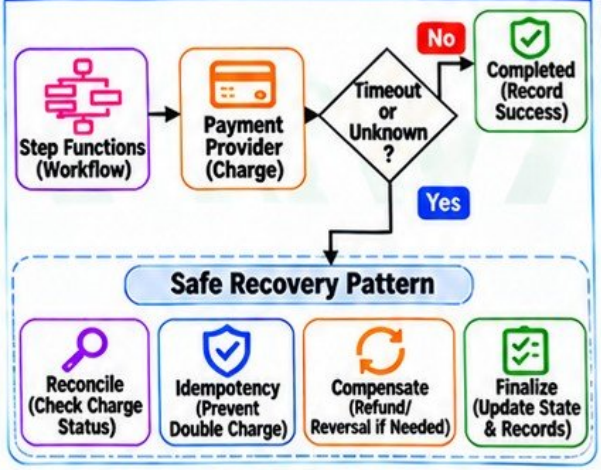
Scenario 30

A Step Functions workflow times out after an external payment provider completes a charge. The workflow cannot determine whether retrying is safe. How would you design reconciliation, compensating actions, and idempotent recovery?

Key Investigation Areas

- Check Step Functions execution history (state, timeout, and last successful step).
- Verify the payment provider's transaction status (using transaction ID).
- Review idempotency design (idempotency key, uniqueness, and request replay behavior).
- Define a reconciliation process (system of record vs, external provider).
- Determine compensating actions (e.g., refund, reversal, or manual review).

Workflow and Recovery Design



Expected Outcome

- Confirm the actual outcome of the payment (charged, not charged, or unknown).
- Use idempotency keys to prevent duplicate charges during retries.
- Run a reconciliation process to align workflow state with the payment provider.
- Apply compensating actions (such as refund or reversal) when required.
- Ensure the workflow reaches a consistent and correct business state.

Senior Interview Focus:

Partial completion, durable state, workflow compensation, and business correctness.

The Multi-Region Architecture That Made the Outage Worse

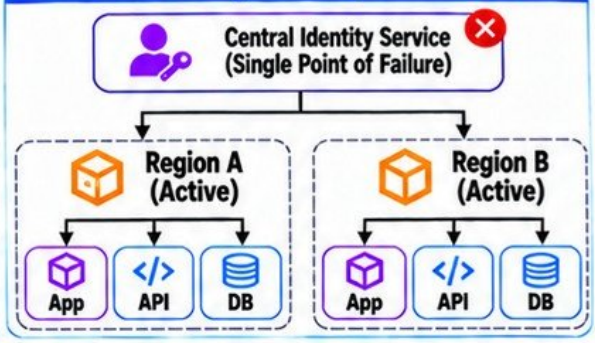
Scenario 31

A company uses active-active application infrastructure across two AWS Regions, but both Regions depend on one centralized identity service. That service fails. How would you identify the hidden shared dependency and redesign the failure domains?

Key Investigation Areas

- Map end-to-end dependencies (app, auth, network, and data).
- Identify shared or centralized services (e.g., identity, KMS, config, or third-party).
- Review architecture, IaC, and service dependency graphs.
- Assess blast radius and cross-region impact.
- Validate alternative identity options (regional or multi-active).

Shared Dependency Problem



Expected Outcome

- Confirm the identity service as the hidden shared dependency.
- Redesign for failure-domain independence (e.g., regional or multi-active identity service).
- Reduce blast radius so one Region can operate independently.
- Test with regional isolation and failover drills.

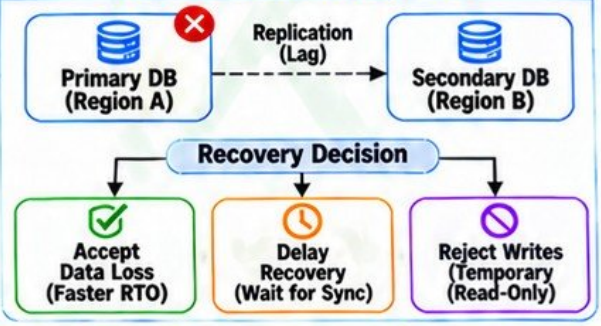
Scenario 32

A regional outage triggers automatic failover, but replication lag means the secondary database is missing recent writes. How would you decide whether to accept data loss, delay recovery, or temporarily reject writes?

Key Investigation Areas

- Measure replication lag (RPO) and write volume.
- Identify which data is missing (and business impact).
- Review RTO, RPO, and business continuity requirements.
- Evaluate options: accept loss, delay switchover, or read-only mode.
- Communicate risk and decision with stakeholders.

Failover Decision Options



Expected Outcome

- Make a risk-based decision aligned to RTO and RPO.
- Understand the business impact of lost or delayed writes.
- Use a controlled recovery (e.g., read-only, then catch up).
- Ensure clear communication and post-incident reconciliation (e.g., replay or backfill).

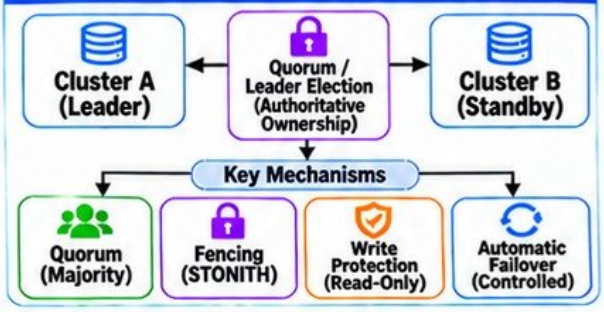
Scenario 33

During a network partition, two database clusters both believe they should accept writes. What mechanisms would you require to establish authoritative ownership and prevent split-brain behavior?

Key Investigation Areas

- Analyze the partition scenario (network, health checks, quorum).
- Review the database's HA model (e.g., quorum, leader election).
- Check fencing and write protection mechanisms (e.g., STONITH, exclusivity locks).
- Validate consistency and recovery behavior after reconnection.
- Test split-brain prevention and failover procedures.

Split-Brain Prevention Design



Expected Outcome

- Establish a single authoritative writer at any time.
- Use quorum and fencing to prevent dual writers.
- Enforce write protection (secondary = read-only).
- Ensure safe recovery and reconciliation after the partition heals.

Senior Interview Focus: Failure-domain independence, RPO versus RTO, and authoritative write ownership.

Backups Exist, but Recovery Still Fails

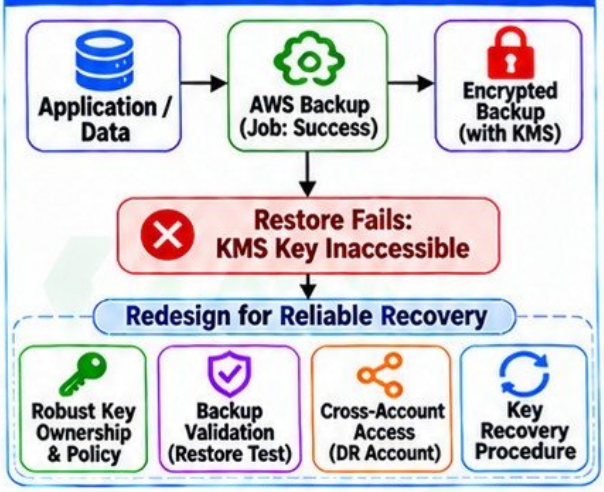
Scenario 34

An AWS Backup job reports success, but a disaster recovery exercise cannot restore the workload because the required KMS key is inaccessible. How would you redesign key ownership, backup validation, and cross-account recovery?

Key Investigation Areas

- Confirm backup encryption (KMS key ID, key policy, and ownership).
- Check KMS key status (enabled, not deleted, and accessible).
- Verify cross-account access (key policy, grants, and IAM permissions).
- Review backup restore logs and failure errors (e.g., AccessDeniedException).
- Assess key recovery and operational ownership (availability and access during DR).

Backup & KMS Dependency Flow



Expected Outcome

- Ensure the KMS key is owned, accessible, and backed up (when applicable).
- Use a multi-account DR design with proper key policies and grants.
- Validate backups regularly with restore tests (not just "job success").
- Document and test a key recovery process (e.g., key replica, escrow, or break-glass access).
- Confirm end-to-end restore works in the DR account or Region.

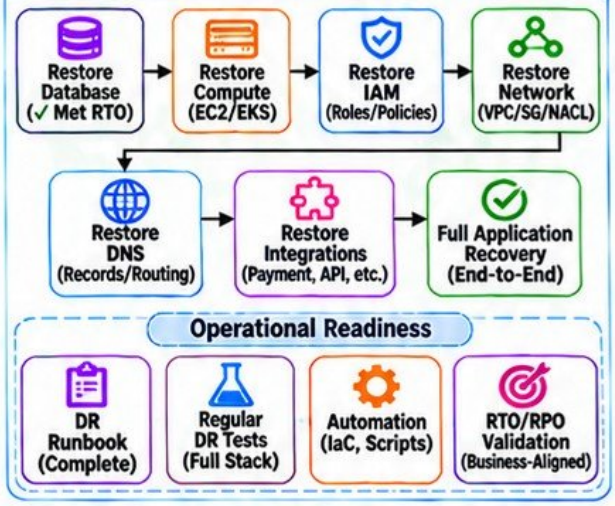
Scenario 35

A company meets its database restore-time target, but restoring the full application takes six additional hours because IAM, DNS, networking, and external integrations are missing. How would you redefine recovery testing and operational readiness?

Key Investigation Areas

- Map the full recovery scope (database, compute, IAM, networking, DNS, and integrations).
- Identify missing dependencies (roles, policies, certificates, VPC/network, DNS records, and third-party services).
- Review the current DR runbook and recovery sequence.
- Measure actual RTO/RPO against business requirements.
- Assess readiness gaps (automation, IaC, and integration recovery).

End-to-End Recovery Flow



Expected Outcome

- Redefine recovery as an end-to-end application restoration (not database only).
- Maintain a complete, tested DR runbook that includes IAM, DNS, networking, and external integrations.
- Use automation and Infrastructure as Code (IaC) to rebuild the full environment quickly and consistently.
- Run regular end-to-end DR exercises and verify RTO/RPO against business objectives.
- Ensure operational readiness with clear ownership, checklists, and evidence of successful recovery.

Senior Interview Focus: End-to-end restoration, encryption dependencies, recovery evidence, and business-level objectives.

Terraform Says Apply, but Production Says Stop

Scenario 36

A Terraform plan unexpectedly proposes replacing a production RDS database after a module update. How would you investigate state drift, provider behavior, immutable attributes, and safe migration alternatives?

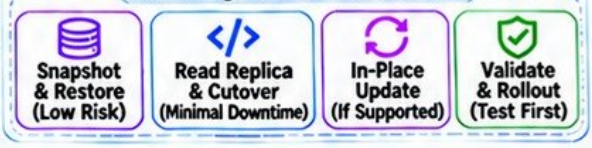
Key Investigation Areas

- Compare Terraform state vs. actual AWS resources (state drift).
- Review the plan diff to see why a replacement is proposed ("-/+" or "forces replacement").
- Check provider and module changes (version, defaults, and behavior).
- Identify immutable RDS attributes that trigger replacement (e.g., engine, storage type).
- Evaluate safe migration options (snapshot, restore, or replica cutover).

Investigation and Decision Flow



Safe Migration Alternatives



Expected Outcome

- Confirm the root cause (state drift, provider change, or immutable attribute).
- Avoid accidental production replacement.
- Choose a safe migration strategy (replica, snapshot, or controlled cutover).
- Update code and state to reflect the final, approved infrastructure.

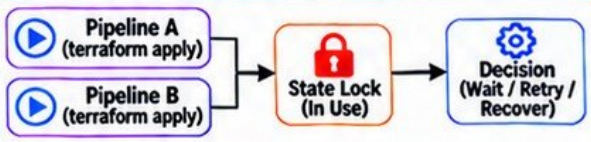
Scenario 37

Two deployment pipelines attempt to modify shared Terraform-managed infrastructure simultaneously. One holds the state lock while the other times out. How would you redesign state boundaries, deployment coordination, and recovery from interrupted operations?

Key Investigation Areas

- Identify the state lock owner and current operation (e.g., S3/DynamoDB lock).
- Review pipeline logs and execution timing (who started first and why).
- Check for partial or interrupted runs (incomplete apply).
- Evaluate state boundaries (one state vs. multiple, by domain).
- Assess coordination strategy (queues, approvals, or serialized runs).

Deployment Coordination Flow



Redesign Options



Expected Outcome

- Prevent concurrent changes to the same infrastructure.
- Use clear state boundaries (e.g., network, database, application) where appropriate.
- Ensure reliable locking and safe recovery from interrupted runs (e.g., force-unlock with validation).
- Implement coordinated deployments (queue, approval, or CI/CD gating).

Scenario 38

An emergency production change is made through the AWS console during an outage. The next Terraform deployment attempts to reverse it. How would you reconcile emergency changes while preserving auditability and infrastructure ownership?

Key Investigation Areas

- Identify the out-of-band change (console change) and who made it.
- Compare actual AWS resources vs. Terraform state (detect drift).
- Assess the business reason and impact of the emergency change.
- Decide whether to keep, revert, or replace with a codified change.
- Capture the change in version control with clear audit records (why, who, when).

Reconciliation Flow



Governance and Ownership



Expected Outcome

- Reconcile the emergency change with Terraform state and code.
- Preserve a clear audit trail (what, why, who, and when).
- Decide to keep, revert, or refactor the change safely.
- Restore IaC ownership and prevent future unmanaged changes.

Senior Interview Focus: State integrity, destructive-change prevention, and controlled infrastructure recovery.

The Deployment Succeeded, but the Release Was Unsafe

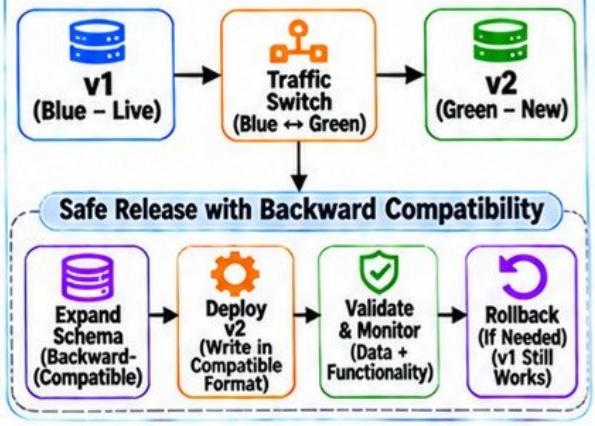
Scenario 39

A blue-green deployment passes health checks, but the new application version writes data that the previous version cannot interpret. How would you design backward-compatible migrations, release sequencing, and rollback authority?

Key Investigation Areas

- Identify the change that breaks backward compatibility (e.g., schema or data format).
- Review the deployment flow (blue-green) and version interactions.
- Check migration strategy (order, safety, and rollback).
- Assess data-read/write compatibility across versions.
- Validate rollback authority (who can rollback, when, and how fast).

Blue-Green & Migration Flow



Expected Outcome

- A release that supports both old and new versions during the transition.
- A safe, backward-compatible data migration strategy (e.g., expand-migrate-contract).
- Controlled release sequencing with validation and quick rollback.
- Clear rollback ownership, criteria, and execution steps.

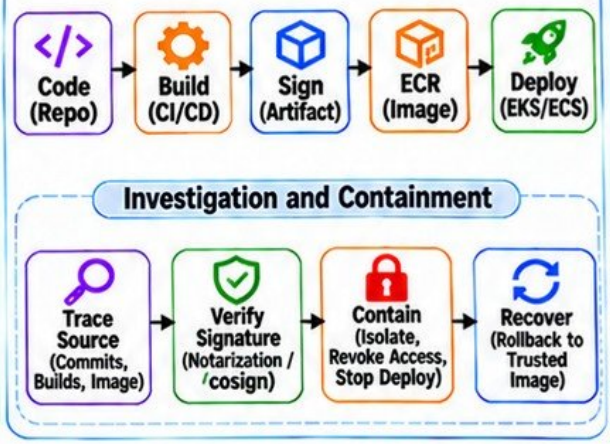
Scenario 40

A compromised CI/CD pipeline publishes a malicious container image to ECR. The image passes functional tests and is deployed. How would you investigate artifact provenance, deployment trust, signing, and containment?

Key Investigation Areas

- Trace the full pipeline (from commit to ECR) and identify the entry point of compromise.
- Verify the artifact's provenance (source code, build logs, and image layers).
- Check if image signing and verification (e.g., cosign) were used and enforced.
- Review pipeline access, secrets, and permissions (IAM, OIDC, roles).
- Assess the blast radius and containment actions (isolation and rollback).

CI/CD and Artifact Trust Flow



Expected Outcome

- The source of the compromise identified and removed (e.g., leaked credential, malicious commit, or runner).
- Verified artifact integrity with image signing and policy enforcement (e.g., cosign + ECR).
- The malicious deployment contained and rolled back quickly.
- A hardened pipeline with least privilege, signed images, and deployment gates (e.g., OPA/Admission Policy).

Senior Interview Focus:

Release safety, artifact identity, supply-chain trust, and irreversible state changes.

CloudWatch Is Green, but Customers Are Complaining

Scenario 41

CloudWatch infrastructure alarms show no issues, yet customers report intermittent checkout failures. How would you investigate distributed traces, application-level errors, dependency latency, and missing service-level indicators?

Key Investigation Areas

- Review end-to-end request flow with distributed tracing (e.g., AWS X-Ray / OpenTelemetry).
- Analyze application logs and error patterns (e.g., 4xx/5xx, exceptions, timeouts).
- Check dependency latency (payment, inventory, database, third-party APIs).
- Validate SLIs/SLOs (e.g., checkout success rate, p95 latency, error rate).
- Correlate customer reports with traces, logs, and metrics (time and region).

Investigation Flow

Customer Reports (Checkout Fails) → Trace (Request Flow) (AWS X-Ray) → Analyze Errors & Latency

Key Signals to Validate (SLIs)

- Success Rate (≥ Target)
- Latency (p95/p99)
- Error Rate (4xx/5xx)
- Dependency Latency (External APIs)

Expected Outcome

- Identify the true failing component (application, dependency, or integration).
- Confirm the customer impact (e.g., checkout success rate drop).
- Add or fix missing SLIs/SLOs (e.g., business transaction success, not just CPU).
- Implement targeted fixes and improve end-to-end observability.

Scenario 42

A service produces millions of unique metric dimensions, dramatically increasing observability costs and making dashboards difficult to use. How would you redesign metric cardinality, tracing, logging, and retention without losing incident visibility?

Key Investigation Areas

- Identify high-cardinality dimensions (e.g., userId, orderId, requestId).
- Review which dimensions are essential for operations and incidents.
- Redesign metrics with controlled dimensions (e.g., service, region, status, dependency).
- Optimize logging (structured, level-based, and sampling).
- Define retention and cost policies (high-value vs. low-value data).

Observability Redesign

High Cardinality (Too Many Dimensions) → Redesign Metrics (Control Dimensions) → Logging (Structured + Sampling) → Retention (Tiered & Cost-Optimized)

Recommended Dimension Strategy

- Keep: service, environment, region, status, and error code.
- Remove/limit: userId, orderId, requestId (use tracing instead).
- Use: distributed tracing for per-request investigation.
- Apply: retention (e.g., 30–90 days) and cost alerts.

Expected Outcome

- Lower observability costs with manageable metrics.
- Dashboards that are clear and actionable.
- Preserve incident visibility using tracing and sampled logs.
- A sustainable metrics and retention strategy.

Scenario 43

A CloudWatch alarm triggers an Auto Scaling action, but additional capacity worsens database latency. How would you identify the actual bottleneck and redesign scaling signals to avoid positive feedback loops?

Key Investigation Areas

- Analyze metric correlations (traffic, CPU, DB connections, latency, queue depth).
- Check database indicators (connections, IOPS, locks, thread/connection limits).
- Validate the scaling policy and thresholds (what triggers it and how often).
- Test the impact of added capacity on downstream dependencies (e.g., database, cache).
- Redesign scaling using business-level signals (e.g., request rate, queue age, latency).

Root Cause & Scaling Redesign

Alarm (Triggers Scale Out) → More Instances (Scale Out) → DB Latency Increases (Bottleneck)

Safe Scaling Design

- Use Better Signals (Request Rate / Queue Age)
- Add Guardrails (Max Capacity, Cooldown)
- Protect Dependencies (DB Connection Limits, Pooling)
- Validate with Testing (Load & Chaos Scenarios)

Expected Outcome

- Identify the real bottleneck (e.g., database connection or IOPS contention).
- Redesign scaling signals to reflect end-to-end health (e.g., latency or queue age).
- Prevent positive feedback loops and downstream overload.
- A stable, cost-efficient, and customer-focused scaling strategy.

Senior Interview Focus: User-facing SLIs, telemetry economics, causal investigation, and safe automation.

The AWS Bill Reveals an Architecture Problem

Scenario 44

AWS costs increase by 70% without a corresponding increase in user traffic. How would you investigate NAT processing, cross-AZ transfers, CloudWatch ingestion, data replication, and idle infrastructure?

Key Investigation Areas

- Analyze Cost Explorer and Cost & Usage Reports (by service, region, and tag).
- Check NAT Gateway costs (processed data and traffic volume).
- Review cross-AZ data transfers (EC2, RDS, ALB, and EKS).
- Evaluate CloudWatch costs (metrics, logs, and traces ingestion).
- Inspect data replication (RDS, Aurora, S3, DynamoDB).
- Identify idle or overprovisioned resources (EC2, EBS, load balancers, and unattached volumes).

Cost Investigation Flow

Analyze Cost Data (Cost Explorer) → Identify Cost Spikes (Services/Region) → Validate Root Causes (Usage, Traffic, Config)

Common Cost Drivers to Check

- NAT Processing (High Data Volume)
- Cross-AZ Transfers (Inter-AZ Data)
- CloudWatch Ingestion (Metrics, Logs, Traces)
- Data Replication (RDS/Aurora/S3/DynamoDB)
- Idle Resources (Unused EC2, EBS, LBs)
- Misconfiguration (Overprovisioning, Auto Scaling, Tags)

Expected Outcome

- A clear breakdown of what is driving the 70% cost increase.
- Confirmed usage patterns for NAT, cross-AZ, and CloudWatch.
- Identification of idle or unnecessary resources.
- A remediation plan with quick wins and long-term cost controls (tagging, right-sizing, and architecture changes).

Scenario 45

Finance requests a 30% infrastructure cost reduction, but the workload has strict latency and availability requirements. How would you prioritize rightsizing, commitment discounts, storage lifecycle changes, and architecture redesign without violating SLOs?

Key Investigation Areas

- Review workload performance and SLOs (latency, availability, error rate).
- Rightsize compute and storage (EC2, EKS, RDS, and EBS) using utilization data.
- Evaluate commitment discounts (Savings Plans and Reserved Instances).
- Optimize storage with lifecycle policies (S3 tiering, EBS type, and snapshot retention).
- Assess architecture changes (e.g., serverless, caching, regional placement, or managed services).
- Estimate cost savings and validate that SLOs remain within targets.

Cost Optimization Approach

Assess Current State (Cost, Usage, SLOs) → Select Optimization Options (Rightsize, RI/SP) → Validate Impact (Performance & SLOs) → Implement in Phases (Monitor & Adjust)

Optimization Options & Examples

- Rightsizing (e.g., downsize idle EC2/RDS, right EBS volume)
- Commitment Discounts (Savings Plans / Reserved Instances)
- Storage Lifecycle (S3 tiering, EBS gp3, snapshot retention)
- Architecture Redesign (e.g., caching (ElastiCache), serverless (Lambda), or managed DB)

Expected Outcome

- At least 30% cost reduction while meeting latency and availability SLOs.
- A prioritized, low-risk execution plan (quick wins, medium-term, long-term).
- Clear cost, performance, and risk trade-offs for each change.
- Ongoing cost governance with alerts, budgets, and regular reviews.

Senior Interview Focus:

Unit economics, cost attribution, performance guardrails, and risk-aware financial decisions.

You Are the Incident Commander. What Do You Do First?

Scenario 46

A production payment platform experiences simultaneous API latency spikes, RDS connection exhaustion, and increasing SQS backlog. The CEO demands an immediate rollback, but the last deployment occurred six hours earlier. How would you establish incident command, identify the likely failure chain, and choose a mitigation?

Key Investigation Areas

- Activate incident command (roles, channel, and comms).
- Triage the blast radius (API, RDS, SQS, dependent services).
- Analyze correlation and failure chain (e.g., DB exhaustion → API latency → SQS backlog).
- Check recent changes (deployments, config, traffic, feature flags).
- Evaluate mitigation options (rollback, scale, throttle, queue management).

Incident Response Flow

Declare Incident (P0/P1) → Form Incident Command (IC, Eng, Ops, Business) → Diagnose Failure Chain (API ⇨ RDS ⇨ SQS)

Mitigation Options

- Rollback (If Linked)
- Scale/Increase Capacity (API/RDS)
- Throttle/Rate Limit (Protect DB)
- Drain/Manage SQS Backlog (Prioritize Jobs)

Expected Outcome

- A coordinated incident response with clear ownership and updates.
- An evidence-based decision (rollback, scale, or throttle) with minimal business impact.
- Stabilized platform (latency, DB connections, and queue within targets).
- A post-incident report with root cause, timeline, and preventive actions.

Scenario 47

A critical IAM policy update causes failures across multiple AWS accounts. Security refuses to restore the previous policy because it closes a known vulnerability. Operations demands immediate recovery. How would you lead the decision, evaluate compensating controls, and document residual risk?

Key Investigation Areas

- Assess the blast radius (accounts, services, and users impacted).
- Validate the policy change (what changed, why, and the security risk addressed).
- Identify critical business impact and time sensitivity (RTO/RPO).
- Evaluate compensating controls (e.g., scoped temporary permissions, WAF, SCPs, or conditional access).
- Capture the risk decision (approval, residual risk, and expiration plan).

Decision and Recovery Flow

Assess Policy Change (Impact & Risk) → Evaluate Options (Restore vs Compensate) → Make Risk Decision (Security + Ops + Business) → Implement & Validate (Controlled Recovery)

Compensating Controls (Examples)

- Scoped Temporary Access (Least Privilege)
- WAF / IP Restrictions (Aisk Reductios)
- Service Control Policies (SCPs) (Guardrails)
- Timpleound Access (with Revlew and Removal)

Expected Outcome

- A balanced decision that restores key services without reintroducing the vulnerability.
- Documented compensating controls and a defined expiry/review date.
- Clear risk ownership and business sign-off.
- A follow-up plan to design a permanent, secure policy with minimal disruption.

Scenario 48

A production Region is degraded, but automatic failover could cause data inconsistency. The business wants zero downtime. How would you communicate the technical constraints, select a recovery strategy, and define the conditions for resuming writes?

Key Investigation Areas

- Assess the regional impact (health, data replication, and consistency state).
- Evaluate recovery options (failover, read-only mode, partial traffic shift).
- Communicate constraints clearly (RPO/RTO, risk of inconsistency, and trade-offs).
- Define resumption criteria (replication caught up, data validated, and health checks).
- Coordinate with business (stakeholders, impact, and communication plan).

Recovery and Communication Flow

Assess Region Health (Degraded, Data Lag) → Evaluate Options (Failover, Read-Only, Partial Shift) → Communicate Trade-offs (Risk, Downtime, Data Consistency) → Execute & Monitor (With Clear Exit Criteria)

Recovery Strategy Options

- Failover (If Data is Safe and Consistent)
- Read-Only Mode (Protect Data Integrity)
- Partial Traffic Shift (Reduce Impact)
- Resume Writes (After Replication Sync & Validation)

Expected Outcome

- Transparent, timely communication with clear risks and options.
- A recovery strategy that preserves data integrity and minimizes downtime.
- Well-defined conditions for resuming writes (e.g., replication in sync and validation complete).
- A documented incident summary and improvements (for resilience and failover).

Senior Interview Focus:

Incident leadership, uncertainty management, risk ownership, and decision-making under pressure.

Final Architecture Defense: Three Failure Domains, One Business

Scenario 49

You inherit an AWS architecture with the following:

Current Architecture (Issues)

- One AWS account for dev, staging, and prod.
- EKS runs all applications.
- One NAT gateway for outbound traffic.
- RDS Multi-AZ for database availability.
- CI/CD has broad production permissions.
- Terraform state is shared across teams.
- Backups exist, but no full restore test.

Your Task (90 Days)

- Identify 5 highest-priority risks.
- Propose a phased remediation plan.
- Explain which changes to postpone.
- Define evidence that proves the architecture is safer.

Key Deliverables (Example)

- Risk list with business impact and priority.
- 0–30/31–60/61–90 day plan.
- Deferred items with clear reasons (cost, risk, or dependency).
- Measurable evidence (e.g., least privilege, restore test report, DR drill results, security and reliability metrics).

Top 5 Priority Risks (Likely)

1. No environment isolation (high blast radius).
2. CI/CD over-privileged access (critical security risk).
3. Shared Terraform state (change and data corruption risk).
4. Single NAT gateway (single point of failure and cost risk).
5. Untested backups (unknown recovery capability).

Phased Remediation Plan (High Level)

0 – 30 Days (Stabilize)

- Separate accounts (Prod / Staging / Dev).
- Restrict CI/CD permissions (least privilege).
- Secure and version Terraform state (locked, remote, access controls).

31 – 60 Days (Harden)

- Add NAT per AZ (or NAT + VPC endpoints).
- Enforce RBAC and Prod controls on EKS.
- Improve monitoring, logging, and alerting.

61 – 90 Days (Validate)

- Run a full backup and restore test (DR drill).
- Document results (RTO/RPO, security, reliability).
- Review costs and optimize without doubling spend.

Scenario 50

You are presenting a new AWS platform architecture to a review board with these business requirements:

Business Requirements

- 99.95% monthly application availability.
- RTO of 30 minutes.
- RPO of 5 minutes.
- Strict separation of production privileges.
- Predictable cloud spending.
- Safe, auditable deployments.
- Recovery from an Availability Zone failure.
- A documented strategy for regional failure.

Your Design and Defense Must Cover

- Architecture Design (end-to-end)
- Service Choices (with trade-offs)
- Cost Drivers (major components)
- Remaining Single Points of Failure (and mitigation)
- How It Meets Requirement (99.95%, RTO, RPO)
- Security & Access Model (separation, least privilege)
- Deployment Strategy (safe, auditable, rollback)
- Evidence & Validation (metrics, tests, DR results)

Expected Outcome

- A resilient, secure, and cost-aware AWS platform.
- Clear justification for key design decisions and trade-offs.
- Identified remaining risks and how to manage them.
- A measurable proof plan (e.g., HA test, AZ failover, DR/region test, RTO/RPO validation, and cost guardrails).

Principal Engineer Test:

Your architecture must survive a technical review from security, SRE, platform engineering, finance, and business leadership.